DisasterShield AI

Project Explainer & Pitch — Kritical Hacks '26

A live-web hazard radar and offline-first survival network for citizens and disaster authorities, built on TinyFish's Search, Fetch and Agent APIs.

This document has two parts: a plain-language explanation anyone can follow, and a formal technical pitch for the offline round. Use Part 1 with family, mentors or non-technical judges; use Part 2 for the jury panel.

PART 1 · IN PLAIN WORDS

What problem are we solving?

When a cyclone or flood hits, two things fail at exactly the wrong moment: people don't get the warning in time, because it's buried on some government website, and once it does arrive, the phone network often goes down too — so even an app that knows what's happening can't tell anyone.

India loses an estimated 12 billion dollars a year to natural disasters, and most of that damage — more than nine in every ten dollars lost — isn't insured or planned for. Floods alone cause roughly two-thirds of that damage.

What does DisasterShield AI actually do?

Think of it as two apps that talk to each other:

- **A citizen app on your phone** that shows live hazard alerts, the nearest shelter, and a big SOS button — and keeps working even with zero signal, because it saves everything it needs in advance.
- **A dashboard for disaster authorities** (like the District Collector's office) that shows where SOS requests are clustering on a map, and which photos of damage sent in by citizens are AI-verified as real versus fake or irrelevant.

How does it know about a hazard before the news does?

We built a “Hazard Radar” that constantly reads official government bulletin pages — IMD weather warnings, flood data, disaster-authority advisories — using a tool called TinyFish. TinyFish does two things for us: it searches the web for the newest alerts, and it fetches and cleans up the text from those official pages so our system can understand them automatically. The radar turns messy webpage text into a simple structured warning (what the hazard is, how severe, which district, and a link back to the original source) and pushes it to every phone in that area.

Why does it still work when the network is down?

The app is built “offline-first.” Before you lose signal, it has already saved the alerts, nearby shelters and map for your area onto your phone. So if a cyclone knocks out the towers, you can still open the app and see what's happening and where to go. If you tap SOS while offline, your location and request are stored

on the phone and sent automatically the moment signal returns — nothing is lost.

How do we stop fake reports from wasting rescuers' time?

When someone uploads a photo of flooding or damage, an AI model checks whether the photo actually shows what it claims, and gives it a confidence score. Authorities see verified, high-confidence reports first, so rescue teams act on real evidence instead of chasing unrelated or duplicate photos.

Who is it for, and who pays for it?

It's completely free for ordinary citizens. It's funded by the institutions that benefit from better data: district governments (a yearly subscription for the authority dashboard), businesses that want a hazard-risk API for their own sites or warehouses, and NGOs doing disaster relief work.

In one sentence

DisasterShield AI turns scattered government hazard data into a warning that reaches you even when the network doesn't — and turns citizen reports on the ground into verified intelligence authorities can act on.

PART 2 · TECHNICAL PITCH (OFFLINE ROUND)

1. Problem Statement

Natural disasters cost India an estimated $12B in 2023 against an $8B decade average (Swiss Re Institute), with floods responsible for roughly 63% of cumulative losses over the past two decades and the large majority of exposure uninsured. The operational failure underlying these losses is twofold: (a) hazard information is fragmented across IMD, CWC, NDMA and state SDMA portals with no unified, machine-readable feed, and (b) last-mile communication collapses precisely when cellular infrastructure is damaged, which is also when citizens most need alerts, routing to shelters, and a way to signal distress.

2. Solution Architecture

The system is a four-service pipeline with a clean JSON contract between each stage:

- **Hazard Radar (Python, TinyFish SDK)** — Search API discovers the latest official bulletins per monitored district; Fetch API renders and cleans the page text; an extraction layer converts free text into a structured alert object (hazard, severity, district, validity window, source URL) and writes it to PostGIS. Raw responses are snapshotted with timestamps for auditability and demo resilience.
- **Citizen PWA (React, Tailwind, Leaflet)** — Service-worker caches the app shell, API responses and map tiles for offline operation. An IndexedDB queue captures SOS triggers and photo reports while offline and flushes them through an idempotent sync endpoint on reconnect.
- **Backend (Node.js, Express, PostgreSQL/PostGIS)** — REST endpoints for alerts, shelters, SOS trigger, offline sync, reports and a grid-based heatmap; JWT-based role separation between citizen and authority users.
- **Vision service (Python, pretrained model)** — Scores uploaded incident photos for label and confidence, so the authority dashboard can rank verified reports above unverified or irrelevant ones.

3. Why TinyFish, specifically

TinyFish provides the live-web layer our radar depends on: Search for fresh, structured results and Fetch for clean page content, both offered at no credit cost, which lets the radar poll continuously without budget risk. For capability that goes beyond read access — e.g. collecting shelter listings from a portal with no public API — the Agent API performs a goal-based automation run, used selectively and cached since Agent and Browser consume credits. This keeps TinyFish central to the product's core claim (live, sourced, auditable hazard data) rather than a decorative integration.

4. Offline-First Design

Offline capability is deliberately separated from the live-data layer: TinyFish supplies freshness while the PWA supplies resilience. The sync loop is idempotent on a client-generated ID, so a retried request after a flaky reconnect cannot create duplicate SOS events. This separation of concerns (sourcing vs. survivability) is the core technical differentiator over broadcast-only alert systems.

5. Impact and Users

- **Citizens** in flood- and cyclone-prone districts — free access to alerts, shelter routing and offline SOS.

- **District Disaster Management Authorities**, chaired by the District Collector under Section 25 of the Disaster Management Act, 2005 — a command dashboard with incident clustering and AI-verified photo evidence.
- **NGOs and businesses** operating in disaster-exposed regions — coordination tooling and a risk-data API.

6. Revenue Model (unit economics)

Stream	Price	Target (Yr 3)
B2G — District SaaS	Rs 6 lakh / district / year	25 districts -> Rs 1.5 Cr
B2B — Risk API	Rs 9,999 - 39,999 / month	40 accounts -> Rs 0.96 Cr
NGO freemium	Free, or Rs 1,999/month Pro	60 orgs -> Rs 0.14 Cr

Year-3 target: ~Rs 2.6 Cr combined revenue. Gross margin per district: ~60% (Rs 3.6L on Rs 6L revenue, against Rs 2.4L cost — cloud, TinyFish credits, SMS fallback and support). Margins hold at scale because the heaviest data-collection load runs on TinyFish Search and Fetch, which carry no credit cost.

7. Honesty on Scope (what's real vs. mocked in this build)

- **Real:** TinyFish Search + Fetch radar, offline PWA with service-worker caching, IndexedDB SOS queue, idempotent sync, pretrained-model photo scoring, PostGIS-backed dashboard with heatmap.
- **Mocked and labelled as such:** SMS gateway (logged, not sent via Twilio), TinyFish Agent runs (executed once and cached rather than live, due to credit cost).
- **Future scope, not in this build:** custom-trained vision model, on-device edge AI, IoT/sensor feeds, drones.

8. Team

- Frontend & UI — Citizen PWA, authority dashboard, offline caching
- Backend & Database — PostGIS schema, SOS/sync API, roles
- AI & GIS — TinyFish radar worker, photo verification model
- Product & Design — Narrative, README, demo video, QA

Sources: Swiss Re Institute (India natural catastrophe economic loss data); TinyFish developer documentation at docs.tinyfish.ai; Disaster Management Act, 2005, Section 25 (District Disaster Management Authority). Revenue figures and market-size estimates are team assumptions, not third-party data.